Web Protocols

Kameswari Chebrolu

Department of CSE, IIT Bombay

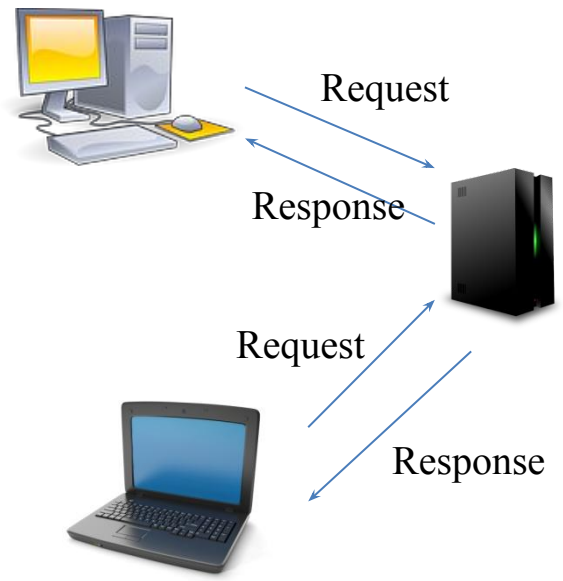
HTTP status ranges in a nutshell:
1xx: hold on
2xx: here you go
3xx: go away
4xx: you messed up
5xx: I messed up

Background: Overall Outline

- ~~What constitutes a webpage?~~
- ~~What goes on inside a Browser?~~
  - ~~What standard security mechanisms implemented?~~
- **How do client and server communicate?**
  - **HTTP/HTTPs protocol; DNS Protocol**
  - Session Management via cookies and tokens
- How does a web server process requests and generate responses?
  - Static vs Dynamic content

Hyper Text Transfer Protocol (HTTP)

- Protocol employed by Web applications
- Based on client-server model
  - Client (browser) requests web objects
  - Server responds with status code and requested object (if present)
- Operates over TCP, server port 80 (http), 443 (https)
- Stateless protocol i.e. no user information stored across requests

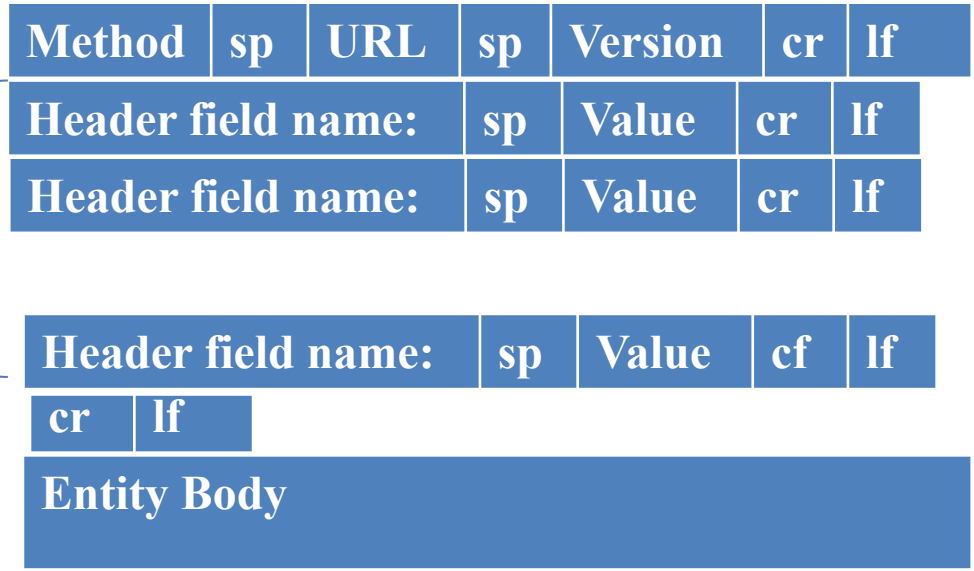


HTTP Message Format

- Two types of messages: Request and Response
- User types http://www.example.com/page.html in browser
- Request Message

GET /page.html HTTP/1.1
Host: www.example.com
User-agent: Mozilla/5.0
Connection: close
Accept-language: En
(blank line)

Header Lines



Methods

Method	Description
GET	Request for a web object
HEAD	Request for header fields (no body); Useful for debugging, get time of last modification etc
PUT	updating or creating a resource; considered idempotent.
POST	submitting data to be processed, often used when user fills forms
DELETE	Remove object
TRACE	Asks server to echo incoming request; Useful for debugging
CONNECT	Used to facilitate secure connection when using Proxy servers
OPTIONS	Query server about its properties or that of an object
PATCH	Similar to PUT except permits partial updating of an object

POST vs GET

- POST requests often involve changing state at server
  - E.g. create user accounts, change user details, invoke system commands, or execute arbitrary code
  - POST based attacks often associated with altering databases or executing commands/code!
- GET requests, on the other hand, just display data
  - GET based attacks often associated with exfiltrating data

Web Forms

- Allows users to upload content to web server in the form of name-value pairs
- Can be achieved both via GET and POST
  - GET (preferable for querying database) http://www.example.com/form.php?name=Chotu
  - POST: name, value is included in the request body
    - Preferable when there is a resulting action, e.g. sending email or writing to database
      - Accidental navigation that triggers GETs does not mess things up!

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Sample Form</title>
</head>
<body>

   <h2>Sample Form</h2>

   <form action="/form.php" method="post">
      <!-- Text input for the user's name -->
      <label for="name">Name:</label>
      <input type="text" id="name" name="name" required>

      <!-- Submit button to send the form data -->
      <input type="submit" value="Submit">
   </form>

</body>
</html>
```

Sample Form

Name: Submit

```
POST /form.php HTTP/1.1
Host: example.com
Content-Type:
application/x-www-form-urlencoded
Content-Length: 10

name=Chotu
```

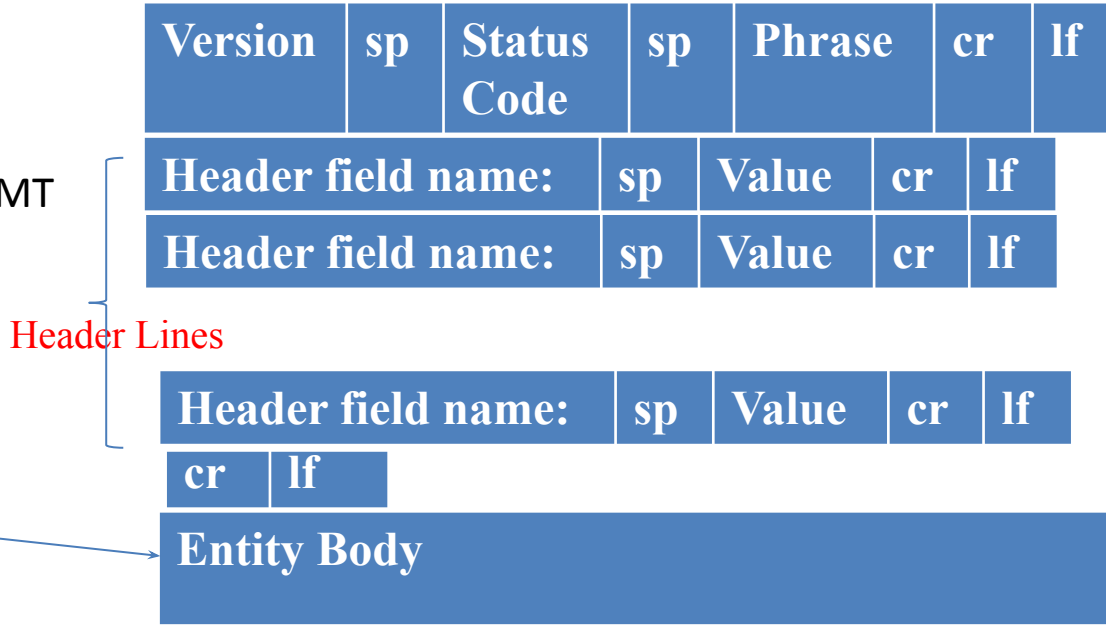
Explanation

- An action attribute set to "/form.php
  - Indicates the URL (endpoint) where the form data will be sent
- Method attribute set to "post," indicating use HTTP POST method
- Label and input elements create a text input field for user to enter name
- Input element with type="submit" creates a submit button
- When user enters name and clicks "Submit" button, a POST request is sent to the "/forms.php" URL

Response Message Format

```
HTTP/1.1 200 OK
Date: Sat, 02 Nov 2013 12:31:22 GMT
Server: Apache
Last-Modified: Fri, 02 Jan 2009 15:55:54 GMT
ETag: "424c001-fd-f9d8d680"
Accept-Ranges: bytes
Content-Length: 253
Content-Type: text/html; charset=UTF-8

data data data data data ...
```



Sample Status Code and Phrases

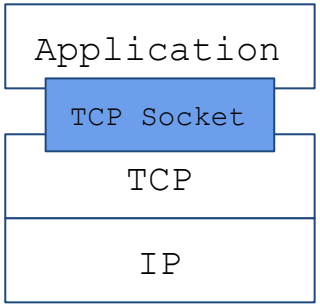
Status Code	Phrase	Description
200	OK	Request successful, information enclosed
301	Moved Permanently	Object moved; new url under Location:
400	Bad Request	Request could not be understood
404	Not Found	Requested object not found on server
503	Service Unavailable	Server is currently unavailable (overloaded)
505	HTTP Version not supported	Server does not support the HTTP version

Hypertext Transfer Protocol Secure (HTTPS)

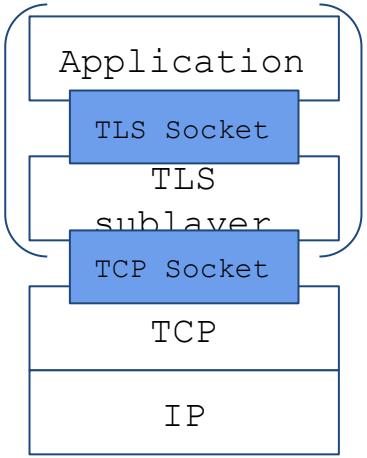
- A secure extension of HTTP
  - URLs that use HTTPS begin with "https://"
    - https://www.example.com
  - Browser connects over TCP to server listening on port 443 (as against 80 for http)
- Work started in 1990s, standardized in 2000s
- 2010s increased adoption due to Let’s Encrypt free certificates
- 2017s major browsers also started pushing “not secure” message □ even more adoption

Implementation

- No changes to HTTP Protocol
- Implemented at transport layer on top of TCP protocol
- Application needs to use TLS API instead of regular socket API



TCP API



Application Layer

TCP enhanced with TLS

Secure Transport Layer

- SSL: originated at Netscape
  - V1 (internal use), v2 (buggy) , v3 (popular but not secure; 2014 poodle attack)
- TLS: standardized by IETF
  - Not compatible with SSL
  - v1.0, v1.1. v1.2, v1.3
  - Evolution accounts for security fixes, newer protocols, removing support for weak protocols

- TLS: Cryptographic protocol authenticates a server to client
  - Also provides confidentiality and integrity
  - Can be used in email, VOIP applications as well!
- Optionally can also
  - Authenticate client to server

- Provides End-to-end security
  - Anything below application sees only encrypted content
    - No router, switch, wifi-AP etc can look at application content
  - Secure even in the presence of an attacker
    - Who owns network
      - Owning → controls routers, switches, Wi-Fi, DNS etc
    - Who can sniff, inject packets

Background

How does TLS provide confidentiality, data integrity and authentication?

A highlevel overview of relevant concepts.

Background

I'VE DISCOVERED A WAY TO GET COMPUTER SCIENTISTS TO LISTEN TO ANY BORING STORY.

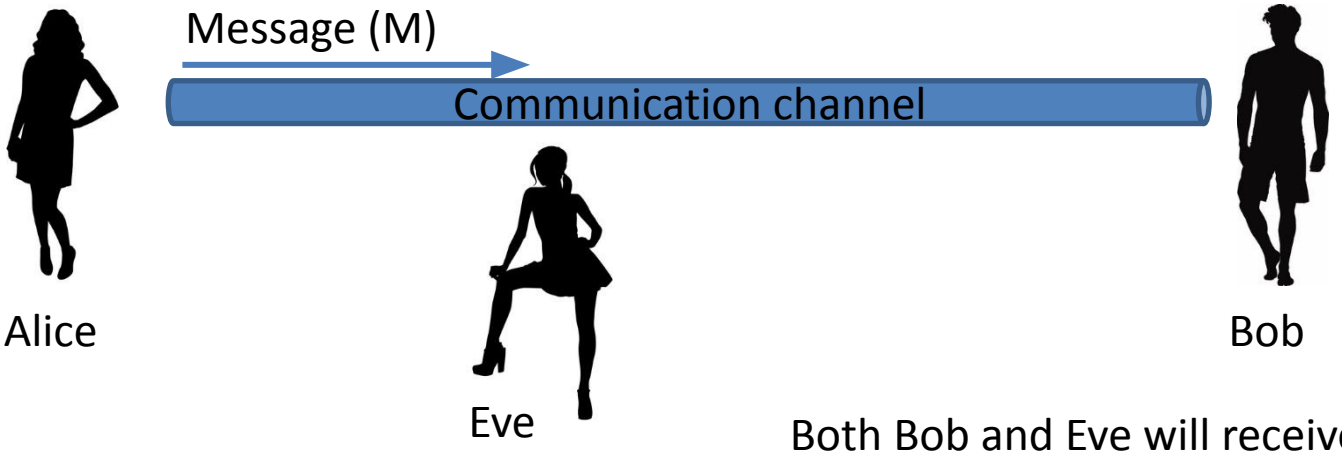
https://xkcd.com/1323/

Players

- Alice (A) and Bob (B) (lovers)
  - In our world: web browser and server
- Eve (E, eavesdropper) (jealous ex)
  - Passive attacker who can listen but not modify messages
- Mallory (M, malicious) or Trudy (T, intruder)
  - Active attacker who can modify, substitute, replay messages
- Goal: Alice and Bob want to communicate **securely** in presence of interlopers like E, M or T
  - Securely: Confidentiality, Integrity (Authentication included)

Confidentiality

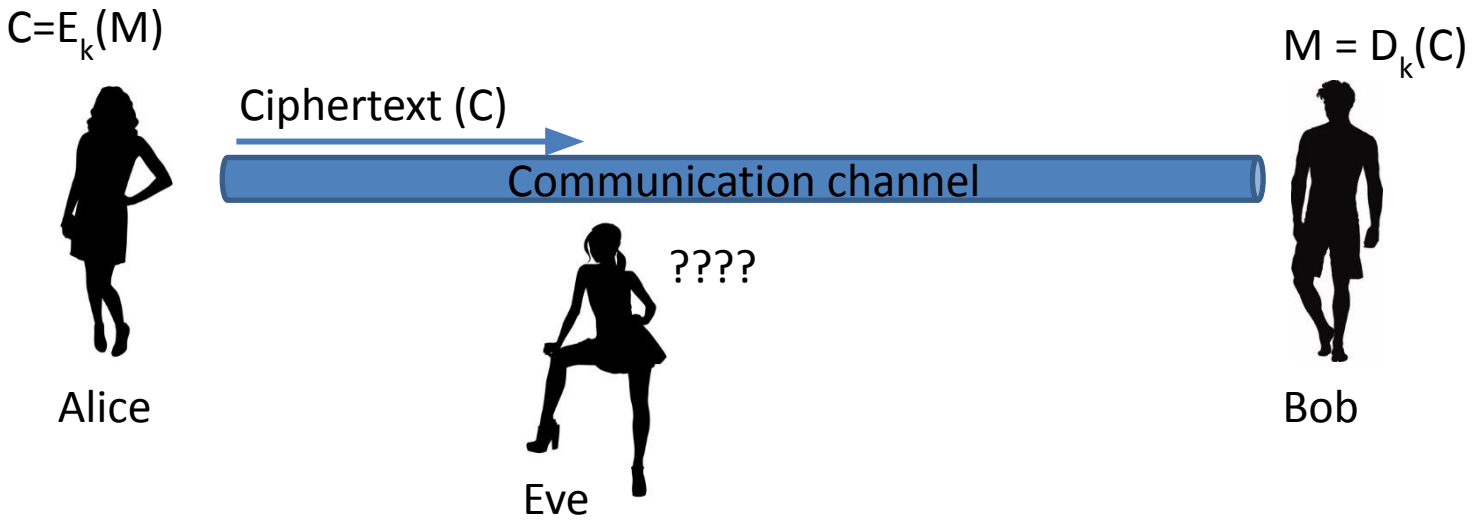
- Information not available or disclosed to unauthorized entities
- Solution: Encryption and Decryption



Both Bob and Eve will receive message

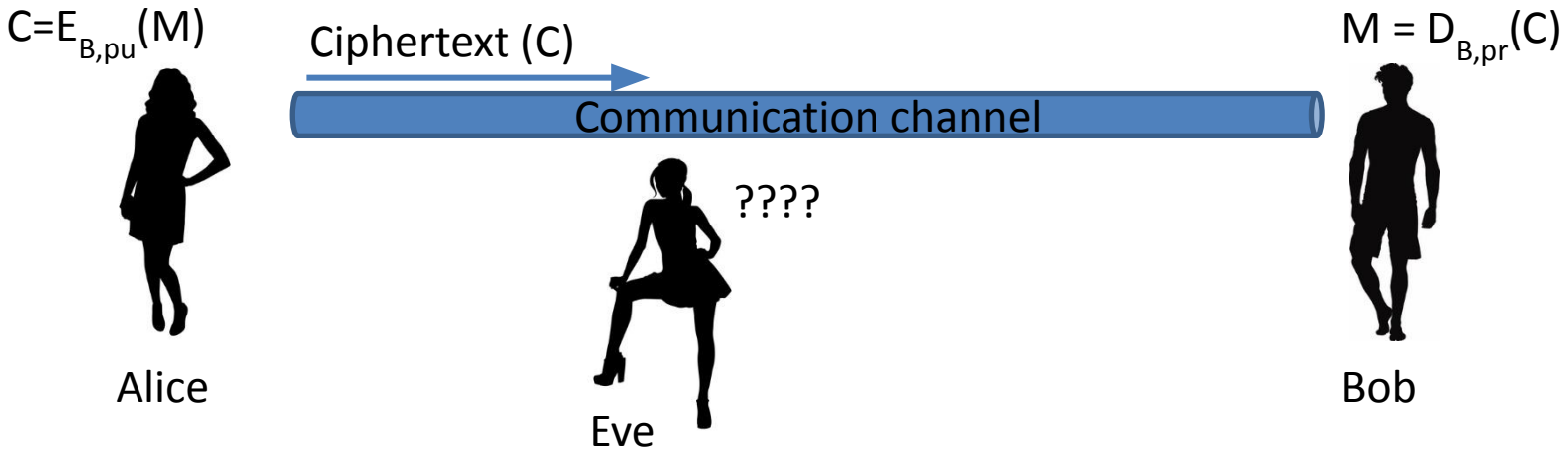
Symmetric Key

- Also called Shared key
  - Alice and Bob share a key k
  - Eve does not know the key but knows the encryption/decryption algorithm
  - E.g. AES, 3DES



Asymmetric Key

- Also called Public-key
  - Bob has two keys: one public and one secret
  - Bob's public key (B,pu) is public, both Eve and Alice have access to this
  - Bob keeps private key (B,pr) secret
  - Alice encrypts message with Bob's public key
  - E.g. RSA, ECC



Integrity and Authentication

- Integrity (data): Has the data been modified in transit?
- Authentication (source): Am I talking with the right person?
- Solutions:
  - Based on Shared key algorithms: Message Authentication Codes (MACs) (e.g. HMAC)
  - Based on Asymmetric key algorithm: Digital signatures (e.g. RSA or ECDSA)

Digital Certificates

- A trusted entity signs some “data” using its private key (public key system)
  - Trusted Entity: Certificate Authority (CA)
  - Data: public key, public key owner identifier, expiration time, serial number etc
  - Signed document: Digital certificate

Example:

- VeriSign (CA) signs a digital certificate for IITB using VeriSign's private key
- Certificate claims www.iitb.ac.in's public key is xyz
- IITB can use this certificate to convince users visiting IITB website that the site is genuine
- User (Browser i.e.) has Verisign's public key (hardcoded) and can verify this certificate!

```
Certificate:
   Data:
       Version: 3 (0x2)
       Serial Number:
           10:e6:fc:62:b7:41:8a:d5:00:5e:45:b6
       Signature Algorithm: sha256WithRSAEncryption
       Issuer: C=BE, O=GlobalSign nv-sa, CN=GlobalSign Organization Validation CA - SHA256 - G2
       Validity
           Not Before: Nov 21 08:00:00 2016 GMT
           Not After : Nov 22 07:59:59 2017 GMT
       Subject: C=US, ST=California, L=San Francisco, O=Wikimedia Foundation, Inc.,
CN=*.wikipedia.org
       Subject Public Key Info:
           Public Key Algorithm: id-ecPublicKey
               Public-Key: (256 bit)
           pub:
                   00:c9:22:69:31:8a:d6:6c:ea:da:c3:7f:2c:ac:a5:
                   af:c0:02:ea:81:cb:65:b9:fd:0c:6d:46:5b:c9:1e:
                   9d:3b:ef
               ASN1 OID: prime256v1
               NIST CURVE: P-256
```

```
        X509v3 extensions:
            X509v3 Key Usage: critical
                Digital Signature, Key Agreement
            Authority Information Access:
                CA Issuers - URI:http://secure.globalsign.com/cacert/gsorganizationvalsha2g2r1.crt
                OCSP - URI:http://ocsp2.globalsign.com/gsorganizationvalsha2g2
            X509v3 Certificate Policies:
                Policy: 1.3.6.1.4.1.4146.1.20
                  CPS: https://www.globalsign.com/repository/
                Policy: 2.23.140.1.2.2
            X509v3 Basic Constraints:
                CA:FALSE
            X509v3 CRL Distribution Points:
                Full Name:
                  URI:http://crl.globalsign.com/gs/gsorganizationvalsha2g2.crl
            X509v3 Subject Alternative Name:
                DNS:*.wikipedia.org, DNS:*.m.mediawiki.org, DNS:*.m.wikibooks.org,
DNS:*.m.wikidata.org, DNS:*.m.wikimedia.org, DNS:*.m.wikimediafoundation.org, DNS:*.m.wikinews.org,
DNS:*.m.wikipedia.org, DNS:*.m.wikiquote.org, DNS:*.m.wikisource.org, DNS:*.m.wikiversity.org,
DNS:*.m.wikivoyage.org, DNS:*.m.wiktionary.org, DNS:*.mediawiki.org, DNS:*.planet.wikimedia.org,
DNS:*.wikibooks.org, DNS:*.wikidata.org, DNS:*.wikimedia.org, DNS:*.wikimediafoundation.org,
DNS:*.wikinews.org, DNS:*.wikiquote.org, DNS:*.wikisource.org, DNS:*.wikiversity.org,
DNS:*.wikivoyage.org, DNS:*.wiktionary.org, DNS:*.wmfusercontent.org, DNS:*.zero.wikipedia.org,
DNS:mediawiki.org, DNS:w.wiki, DNS:wikibooks.org, DNS:wikidata.org, DNS:wikimedia.org,
DNS:wikimediafoundation.org, DNS:wikinews.org, DNS:wikiquote.org, DNS:wikisource.org,
DNS:wikiversity.org, DNS:wikivoyage.org, DNS:wiktionary.org, DNS:wmfusercontent.org,
DNS:wikipedia.org
```

```
        X509v3 Extended Key Usage:
            TLS Web Server Authentication, TLS Web Client Authentication
        X509v3 Subject Key Identifier:
            28:2A:26:2A:57:8B:3B:CE:B4:D6:AB:54:EF:D7:38:21:2C:49:5C:36
        X509v3 Authority Key Identifier:
            keyid:96:DE:61:F1:BD:1C:16:29:53:1C:C0:CC:7D:3B:83:00:40:E6:1A:7C
```

```
Signature Algorithm: sha256WithRSAEncryption
     8b:c3:ed:d1:9d:39:6f:af:40:72:bd:1e:18:5e:30:54:23:35:
     ...
```

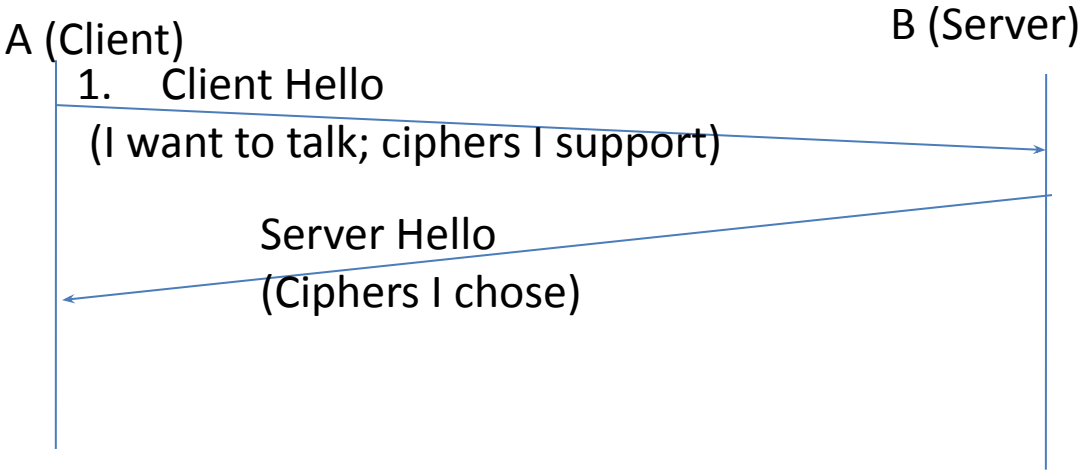
From https://en.wikipedia.org/wiki/Public_key_certificate

Example

- www.iitb.ac.in uses http, say IITB wants to move to https (access via TLS)
- IITB needs to buy a certificate from a CA
  - Links www.iitb.ac.in (domain) with a public key
- When a user types https://www.iitb.ac.in
  - Browser asks IITB's website for a copy of its certificate
  - Browser verifies certificate using public key of CA who issued the certificate
  - Checks domain in certificate matches url typed in browser
  - Establishes secure connection to IITB web server using public key in the certificate

TLS Handshake: Step-1

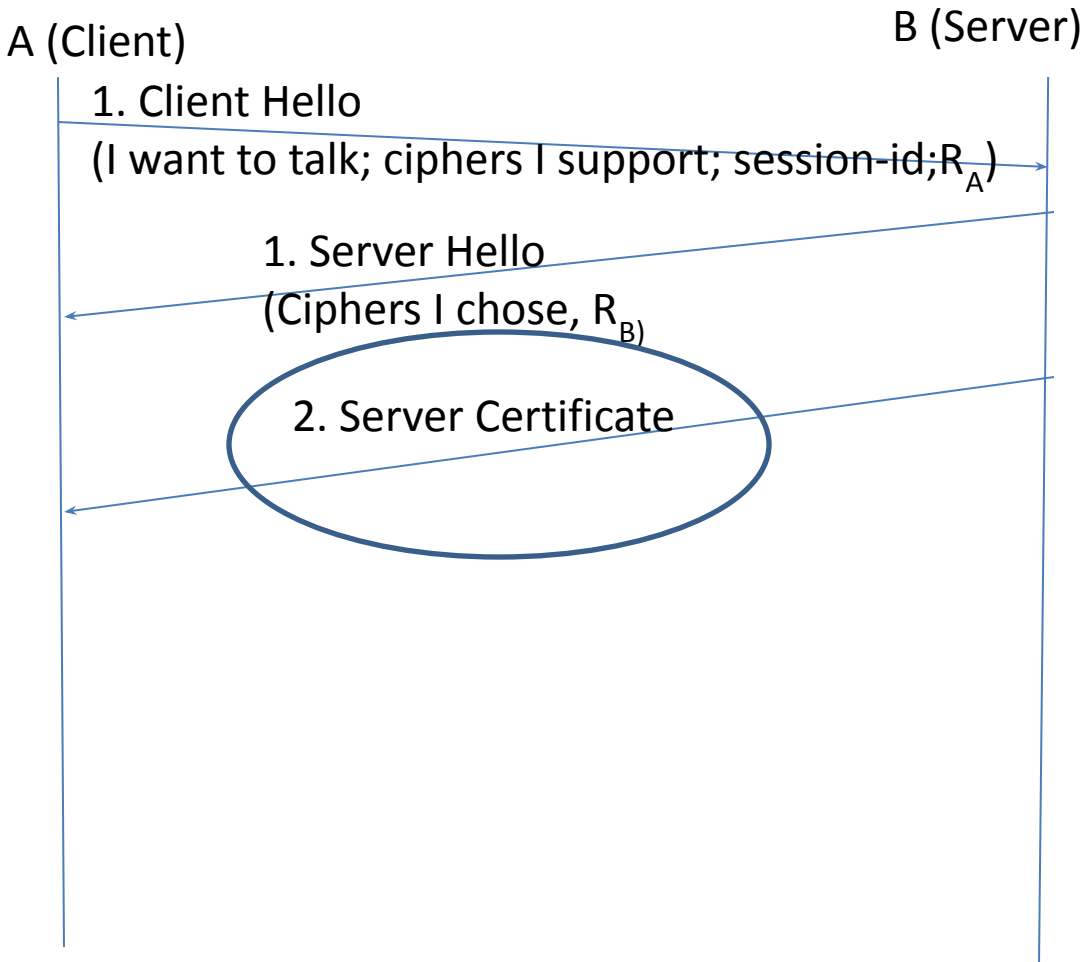
TCP 3-way handshake Precedes the first message



Ciphers I chose: AES for symmetric key, RSA for public key, HMAC for MAC etc

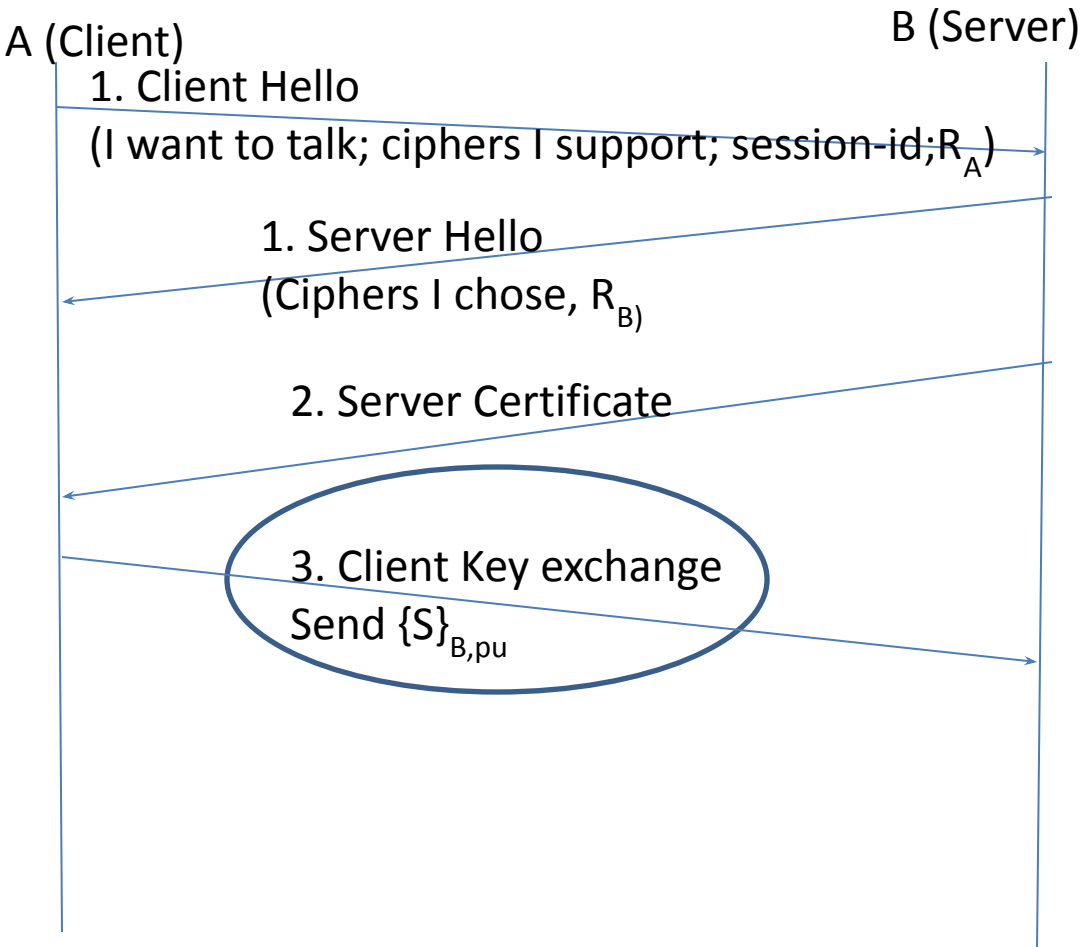
Handshake: Step-2

- Server sends its digital certificate
- Verified by the client using CA's public key



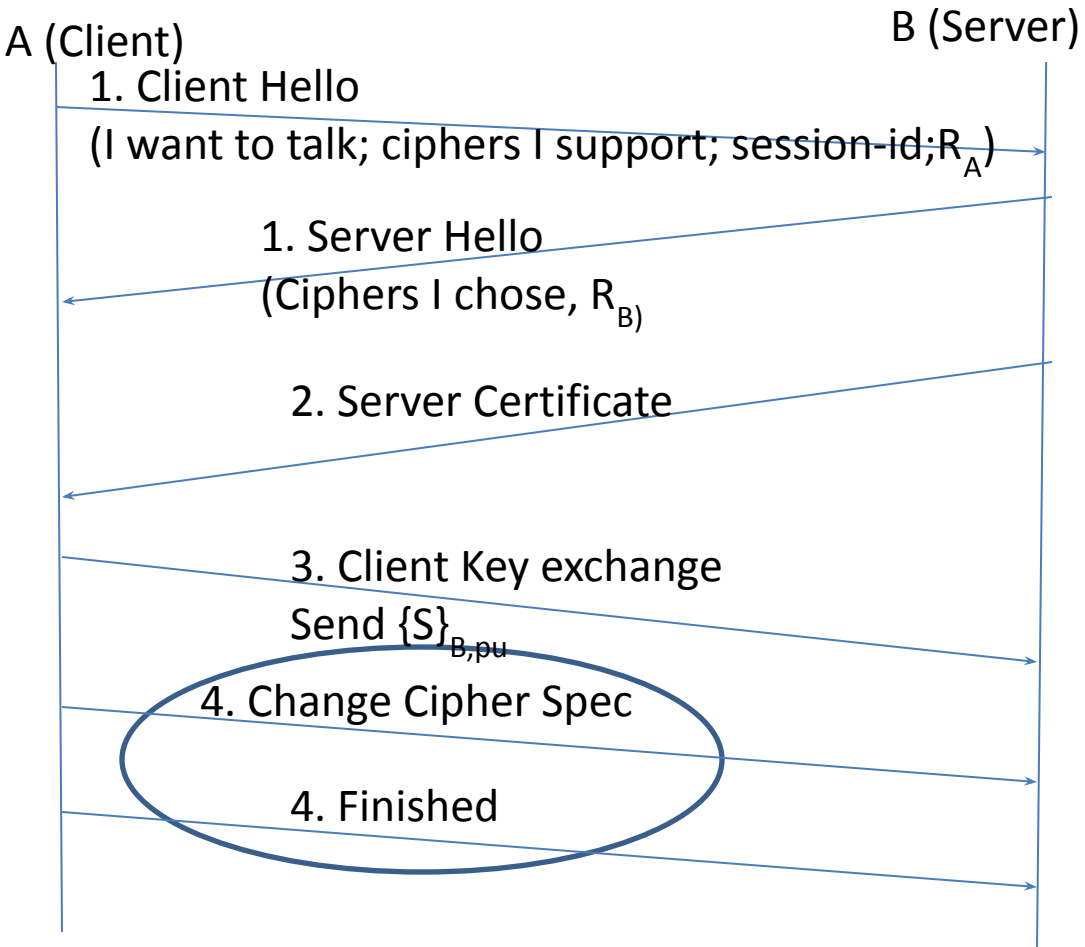
Handshake: Step-3

- Client chooses a random number S
- Encrypts it with server's public key and sends to server
- From "S" client and server computer other keys
  - Needed for providing confidentiality and integrity
  - These are symmetric key based since it is faster!



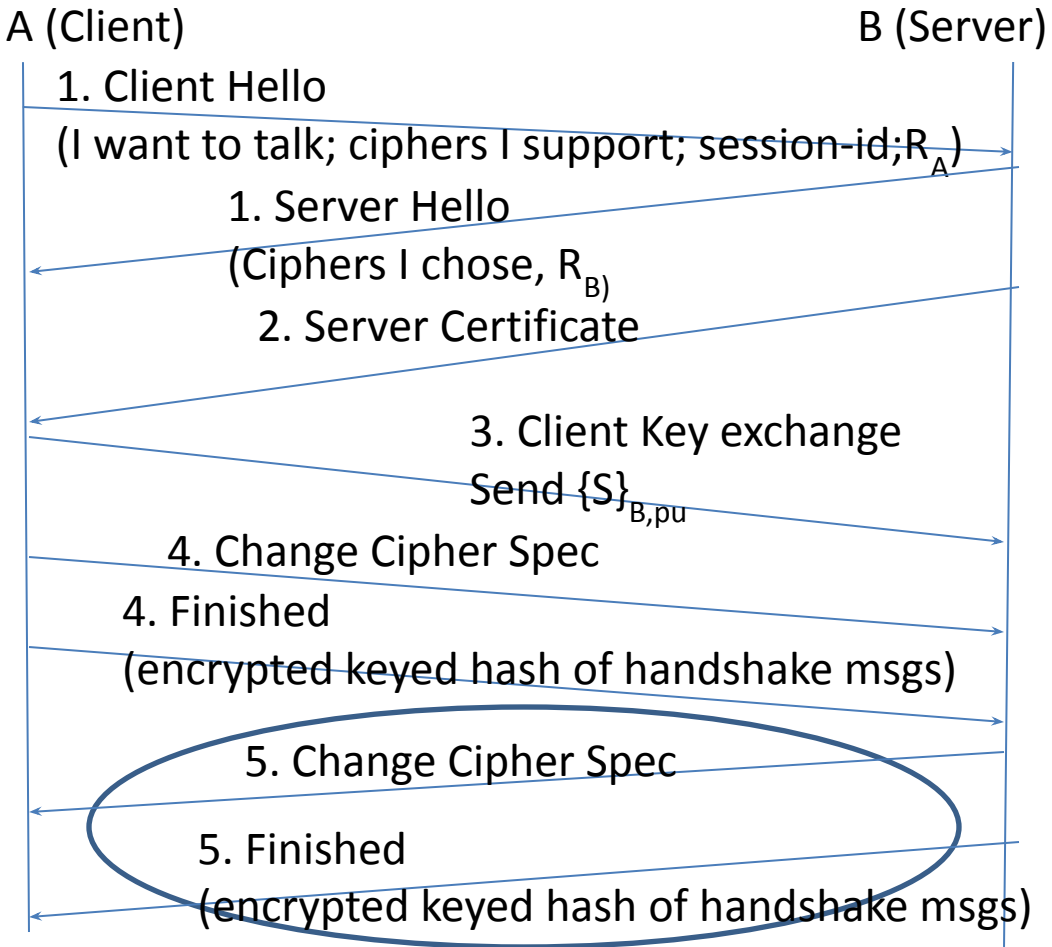
Handshake: Step-4

Now on everything will be encrypted/integrity protected with chosen ciphers and derived keys



Handshake: Step-5

- Server also confirms change of cipher spec
- Now on chosen ciphers and derive keys will be used

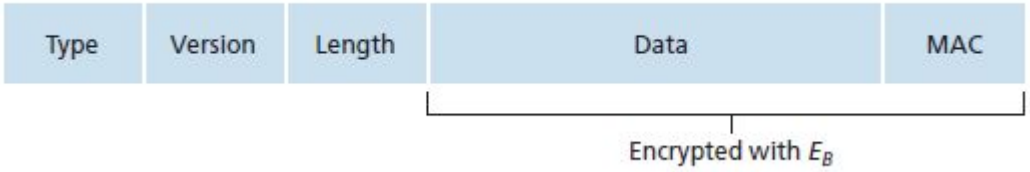


Client Authentication?

- So far, client authenticated the server
- How does server authenticate the client?
  - SSL/TLS does support public key based client authentication (this is optional)
    - Clients having public keys not very realistic
  - Common case: Client send password as part of the data (encrypted) that follows authentication

Data Transfer

- Break data (HTTP messages) and place in records
- Append MAC
- Encrypt (record+MAC), add TLS headers
- Pass above to TCP



Record format

HTTP/2.0

- Second major version standardized in May 2015
- Key features:
  - Multiplexing: allows multiple requests and responses to be sent in parallel
  - Header Compression: minimizes amount of data transmitted
  - Binary Protocol: Compact representation of data

- Server Push: Push resources to clients before they are explicitly requested
  - Can load related resources (such as stylesheets, images, or scripts) proactively
- Stream Prioritization: enables more important resources to be transmitted first
- Backward Compatible with HTTP/1.1
- Many major browsers support HTTP/2 only over secure connections (HTTPS)

HTTP/3.0

- Third major version to enhance performance and security, RFC 9114 on 6 June 2022
- Uses QUIC (Quick UDP Internet Connections) instead of TCP
  - More efficient and faster than TCP
    - Supposedly 3x times faster than HTTP/1.1
  - Built-in support for encryption, flow control, and error correction
- Backward compatible with HTTP/2 and HTTP/1.1
- An evolving standard, slowly gaining adoption

Domain Name System (DNS)

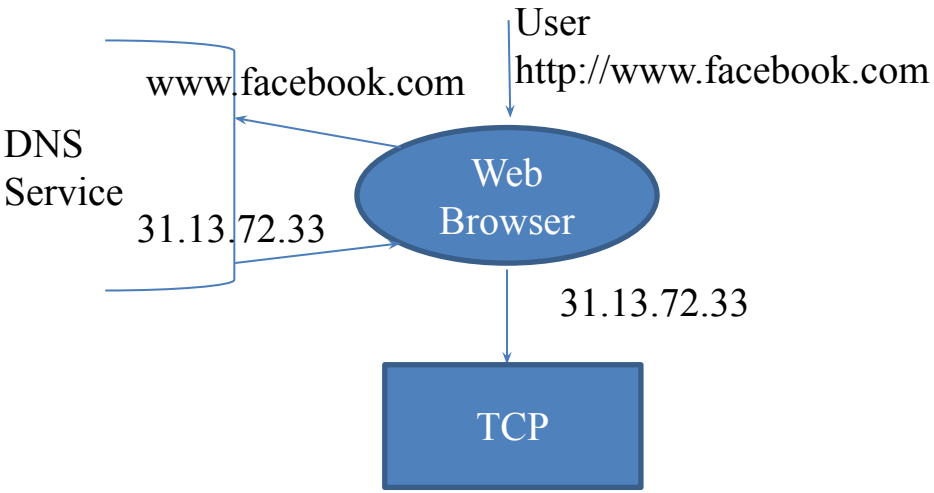
- People have many identifiers: Full name, Pet name, Roll number, Passport number
- Internet hosts are no less
  - Hostnames and IP addresses

E.g. www.facebook.com
- Variable Length
- Mnemonic
- Carry no info to help route packets towards them

E.g.31.13.72.33
- Fixed Length
- Numeric
- Routing information embed within them

Problem and Solution

- People prefer hostnames
- Routers prefer IP addresses
- Need a service (DNS) that converts hostnames/domains to <u>Values</u>



Domain Name: Label that defines a realm of administrative autonomy
E.g. facebook.com; iitb.ac.in; mit.edu

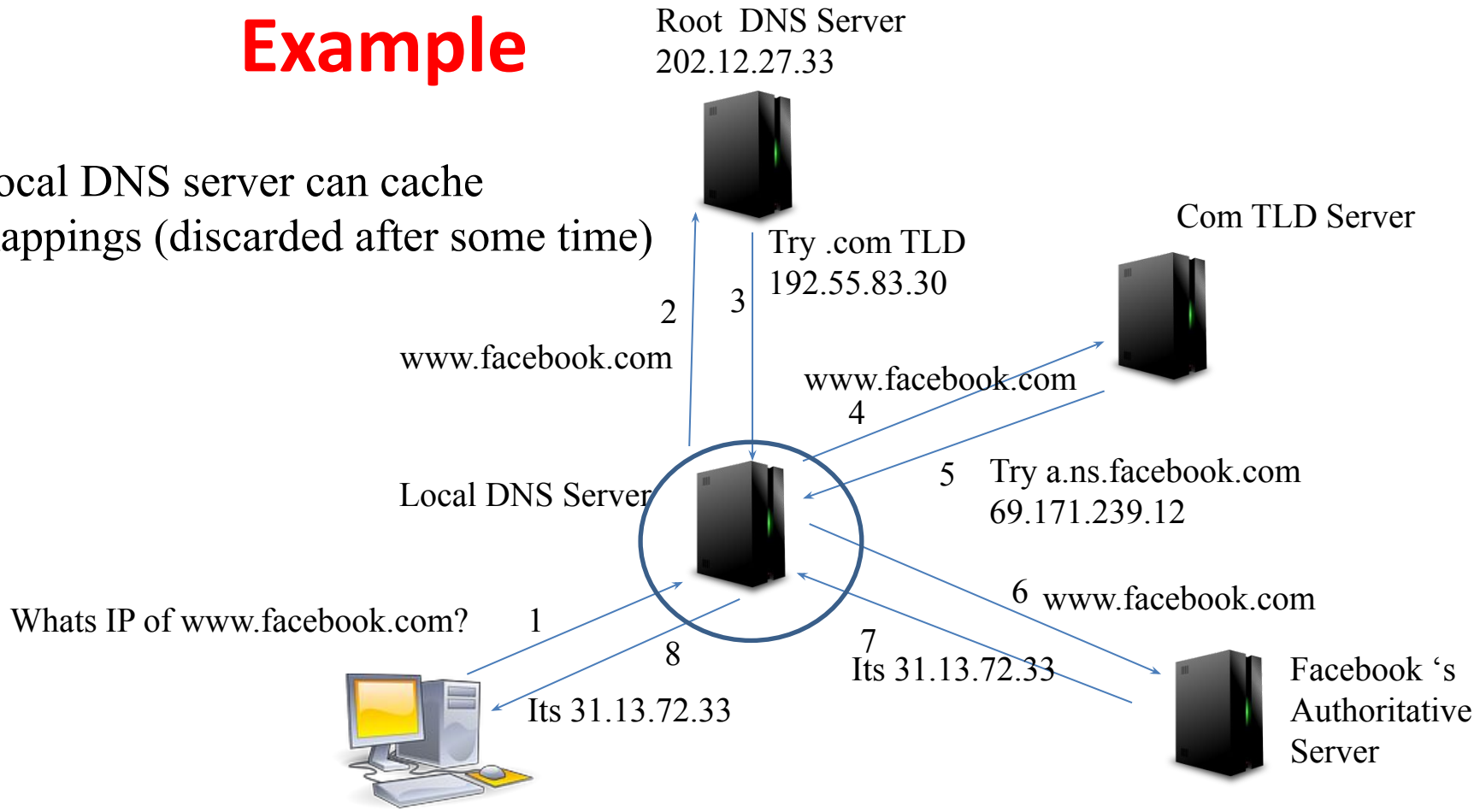
DNS Services

- Host name to IP address translation
  - One hostname can map to multiple IP addresses also
- Host aliasing: many names may map finally to same IP address
  - www.facebook.com. (alias hostname) maps to star.c10r.facebook.com. (canonical hostname)
  - www.facbook.com, www.facebok.com map to www.facebook.com

- Mail server aliasing: Help specify mailserver of a given domain
  - E.g. cse.iitb.ac.in maps to jeeves.cse.iitb.ac.in
  - facebook.com maps to msgin.t.facebook.com
- Load distribution: Helps distribute load across replicated servers
  - A single hostname associated with many IP addresses; order rotated on each request

Example

Local DNS server can cache mappings (discarded after some time)



DNS Server Database

- Store Resource Records (RRs)
- Four Tuple: [Name, Value, Type, TTL]
- Type=A; Name: Hostname; Value: IP Address
  - E.g. [star.c10r.facebook.com, 31.13.72.33, A, 17]
- Type=NS; Name: Domain; Value: host-name of the authoritative name server
  - E.g. [facebook.com, a.ns.facebook.com, NS, 172797]

DNS Database

- Type=CNAME; Name: Hostname; Value: Canonical hostname
  - E.g. [www.facebook.com, star.c10r.facebook.com, CNAME, 2362]
- Type=MX; Name: Hostname; Value: Canonical name of the mail server
  - E.g. [facebook.com, msgin.t.facebook.com, MX, 300]

Summary

- Basics of HTTP/1.1 Protocol
  - message format, methods and response codes
- HTTPs via TLS
  - Steps involved in TLS handshake
- Features of more recent versions (HTTP/2.0 and HTTP/3.0)
- Basics of DNS protocol

References

- TLS1.0: http://www.moserware.com/2009/06/first-few-milliseconds-of-https.html
- TLS1.2: https://medium.com/@ethicalevil/tls-handshake-protocol-overview-a39e8eee2cf5
- HTTP: https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
- HTTP/2.0: https://dl.acm.org/doi/fullHtml/10.1145/2534706.2534721
- QUIC Transport Protocol: Design and Internet-Scale Deployment: https://dl.acm.org/doi/10.1145/3098822.3098842
- DNS: https://www.geeksforgeeks.org/domain-name-system-dns-in-application-layer/